

1. What is `NPM`?

-> 1. NPM is the world's largest software registry, through which developers share and borrow packages, It's an open source. We can also call it database of JavaScript packages.

2. "NPM is the default package manager for Node.js used to install, manage, and handle dependencies and packages in JavaScript projects."

3. It is used to:

- install packages/libraries
- manage project dependencies
- run scripts in projects

2. What is `Parcel/Webpack`? Why do we need it?

-> 1. Parcel and Webpack are **module bundlers** used in JavaScript applications.

2. "Parcel and Webpack are JavaScript bundlers that combine and optimize project files for the browser. They help improve performance, manage dependencies, and support modern frontend development features like hot reloading and code optimization."

3. Browsers cannot efficiently manage large modern applications with many files and dependencies.

Parcel/Webpack help by:

- bundling files
- optimizing code
- improving performance
- enabling modern JavaScript features

3. What is `.parcel-cache`?

-> 1. ".parcel-cache is a cache folder created by Parcel bundler to store temporary build data and improve build performance by avoiding unnecessary recompilation."

2. .parcel-cache is a folder automatically created by Parcel. It stores cached build files to make future builds faster.

3. When Parcel builds your project for the first time, it processes:

- JavaScript
- CSS
- Images
- Dependencies

Instead of rebuilding everything again, Parcel stores temporary processed data inside `.parcel-cache`.

4. This helps in:

- faster rebuilds
- improved development speed
- better performance

5. This folder is temporary and automatically generated. So no need to push it to GitHub.

4. What is ``npx`` ?

-> 1. "npx is a tool used to execute Node packages directly without globally installing them. It helps run packages temporarily and is commonly used for creating projects and running CLI tools."

2. It is used to execute packages directly without globally installing them.

3. Instead of installing packages globally, npx:

- downloads the package temporarily
- runs it
- removes it after execution (if not already installed)

This saves storage and avoids global package installation.

5. What is difference between ``dependencies`` vs ``devDependencies`` ?

-> 1. "dependencies are packages required for the application to run, while devDependencies are packages needed only during development and testing."

2. Both are stored inside `package.json` in a Node.js project, but they serve different purposes.

3. dependencies are packages required for the application to run in production.

Eg. React , Express ,Axios

4. devDependencies are packages needed only during development

Eg. Parcel , Webpack ,ESLint ,Jest.

6. What is Tree Shaking?

->1. "Tree Shaking is a process used by bundlers to remove unused code from the final bundle, which reduces bundle size and improves application performance."

2. we can also say like Tree Shaking is an optimization technique used by bundlers like Webpack and Parcel to remove unused code from the final bundle.

3.It helps reduce bundle size and improve application performance.

7. What is Hot Module Replacement?

-> 1. "Hot Module Replacement is a feature that updates only the changed modules in the browser without reloading the entire page, improving development speed and preserving application state."

2. Hot Module Replacement (HMR) is a feature in bundlers like Webpack and Parcel that updates changed modules in the browser without fully refreshing the page.

3. Benefits of HMR

- Faster development
- Instant updates in browser
- Better developer experience
- Saves development time

8. List down your favourite 5 superpowers of Parcel and describe any 3 of them in your own words.

- 1. HMR : Faster development
- 2. Treeshacking : Removes unused nodes.
- 3. File Watcher Algorithm: This algorithm continuously watch the changes in the file and keep a track.

4. Bundling : Making code compact by removing all spaces and bundle it for production ready.
5. Minify Code : Minify the code and make it production ready.
6. Zero Config : beginner friendly. No need to complex configuration.

9. What is `.gitignore` ? What should we add and not add into it?

-> 1. `.gitignore` is a file where we can write the files and folder names which we don't want to push on Git.

2. We should add the heavy folders like `node_modules`. Which we can install whenever we require. Also we can ignore `/dist` and `.parcel_cache`.

3. we should not add `package.json` and `package-lock.json` files in `.gitignore`.

10. What is the difference between `package.json` and `package-lock.json` ?

-> 1. `package.json`: when we run `npm init` `package.json` file created.

In this file we have all the dependencies and packages which we installed in a Application

2. `package-lock.json`: In this file we get the exact detail versions of packages which we used in our project.

3. "package.json contains project metadata and dependency information, while package-lock.json stores the exact installed package versions to ensure consistent dependency installation across environments."

package.json

Stores package information

Manually editable

Uses version ranges (^, ~)

Defines project dependencies

package-lock.json

Stores exact installed versions

Auto-generated by npm

Uses exact versions

Locks dependency tree

11. Why should I not modify `package-lock.json` ?

-> 1. Package-lock.json is the exact details of packages install, which is require to work the app same on all the systems whenever any developer install it, modification create app may behave differently.

12. What is `node\_modules` ? Is it a good idea to push that on git?

-> 1. node_modules is a folder which contains the all dependencies and sub dependencies which require for the project.

2. This folder is very large in size. So we never push it on git. As and when needed we can reinstall this folder with the help of package.json file.

13. What is the `dist` folder?

-> 1. /dist folder is created when we run npm build for the production.

2. It contains the minified code, which is lighter and ready to deploy.

14. What is `browserslist`

-> 1. Browserslist is a configuration tool used to specify which browsers an application should support.

2. Different browsers support different JavaScript and CSS features, Browserslist tells build tools 1.which browser to target. 2. what code transformations are needed.

3. internally Browserslist configuration Babel transpiles JavaScript, Autoprefixer adds CSS prefixes, And bundler optimize compatibility.

15.Read about dif bundlers: vite, webpack, parcel

--> "Webpack, Parcel, and Vite are frontend build tools/bundlers. Webpack is highly customizable, Parcel focuses on zero configuration, and Vite is a modern fast build tool with extremely fast development server and HMR."

Feature	Vite	Webpack	Parcel
Type	Build Tool + Bundler	Module Bundler	Zero-config Bundler
Speed	Very Fast	Slower compared to Vite	Fast

Feature	Vite	Webpack	Parcel
Configuration	Minimal config	Requires more configuration	Zero or very little config
Development Server	Extremely fast HMR	Good HMR but slower	Fast HMR
Build Process	Uses ES Modules during development	Bundles everything before serving	Automatic bundling
Learning Curve	Easy	Medium to Hard	Easy
Best For	Modern frontend apps	Large enterprise applications	Beginners & medium projects
Setup Complexity	Simple	Complex	Very Simple
Plugin Ecosystem	Growing	Very Large	Smaller than Webpack
Performance	Excellent	Good	Good
Default in Modern Projects	Commonly used today	Older industry standard	Less commonly used now
React Support	Excellent	Excellent	Excellent

16.Read about: ^ - caret and ~ - tilda

^ caret → Automatic upgrade to the minor version

Eg. React 18.0.0 to 18.0.1

~ tilda → Automatic upgrade to the major version

Eg. React 18.0.0 to 19.0.0

Recommended use ^ caret version.

17.Read about Script types in html (MDN Docs)